

¿Qué significa "No modificar la firma del método"?

En el desarrollo de software profesional, la **firma de un método** (*method signature*) es el documento de identidad de una función. Es el "contrato" que el método firma con el resto de la aplicación que quiere utilizarlo.

Fíjate que en Java, la firma de un método está compuesta **única y exclusivamente** por dos cosas:

1. **El nombre del método** (en nuestro caso, `cT`).
2. **La lista de sus parámetros**, teniendo en cuenta su **número**, su **orden** y sus **tipos de datos** (`double`, `int`, `boolean`).

Anatomía de la firma en nuestro código legado

Observemos la cabecera de nuestro método original:

```
public double cT(double m, int tC, boolean dV)
```

En este caso, la firma del método que la máquina virtual de Java (JVM) registra es:
cT(double, int, boolean)

¿Qué NO forma parte de la firma?

Esto suele confundir. **No** forman parte de la firma de un método:

- **El tipo de retorno** (en este caso, `double`).
- **Los modificadores de acceso** (como `public`, `private` o `protected`).
- **Las excepciones** que pueda lanzar el método (cláusula `throws`).

¿Por qué es crucial para la práctica y los tests?

Y es que, si decides cambiar manualmente la firma del método en **FacturacionLegacy.java** (por ejemplo, cambiándole el nombre a **calcularTotal** o cambiando el orden de los parámetros), ocurrirá un desastre al instante:

```
// Si el alumno lo cambia manualmente a esto:  
public double cT(double m, int tC, boolean dV)
```

La clase de pruebas **FacturacionLegacyTest.java** (nuestra red de seguridad) intentará seguir llamando al método antiguo tal y como estaba diseñado:

```
// El test buscará esto y dará un Error de Compilación instantáneo:
```

```
facturacion.cT(100.0, 1, true);
```

Como la firma original `cT(double, int, boolean)` ya no existe en el código, **el proyecto entero dejará de compilar**.

La magia de la Refactorización con el IDE

Aquí es donde entra la lección de oro de la práctica: **¿Cómo cambiamos entonces el nombre a algo más bonito sin romper los tests?**

En lugar de cambiar los nombres de forma manual ("a lo bruto"), debemos delegar esta tarea en el propio entorno de desarrollo. Dependiendo de las herramientas que utilicen en el taller, los atajos de teclado para aplicar un **cambio de nombre seguro (Rename Refactoring)** son los siguientes:

- **Visual Studio Code:** Pulsar Shift + F6 (o clic derecho > *Rename Symbol*).
- **NetBeans:** Pulsar Ctrl + R (o clic derecho > *Refactor* > *Rename*).
- **IntelliJ IDEA:** Pulsar Shift + F6 (o clic derecho > *Refactor* > *Rename*).
- **Eclipse:** Pulsar Alt + Shift + R (o clic derecho > *Refactor* > *Rename*).

Al utilizar cualquiera de estas herramientas de refactorización automatizadas:

1. El IDE cambiará el nombre `cT` por `calcularTotal` en la clase de facturación.
2. **Automáticamente y al mismo tiempo**, el IDE buscará todas las llamadas en la clase de pruebas y las actualizará por ti a `facturacion.calcularTotal(100.0, 1, true)`.

Lo importante aquí es que comprendan que **los contratos (las firmas) se respetan**, y que si necesitamos alterarlos, debemos usar las herramientas quirúrgicas del IDE en lugar del borrado manual.

